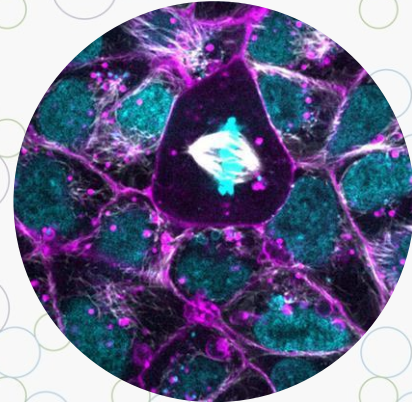




Basic Python Part 2 - Control Flow and Functions

Coding with Allen Institute Data: A Python Workshop for Undergraduate Neuroscience Educators

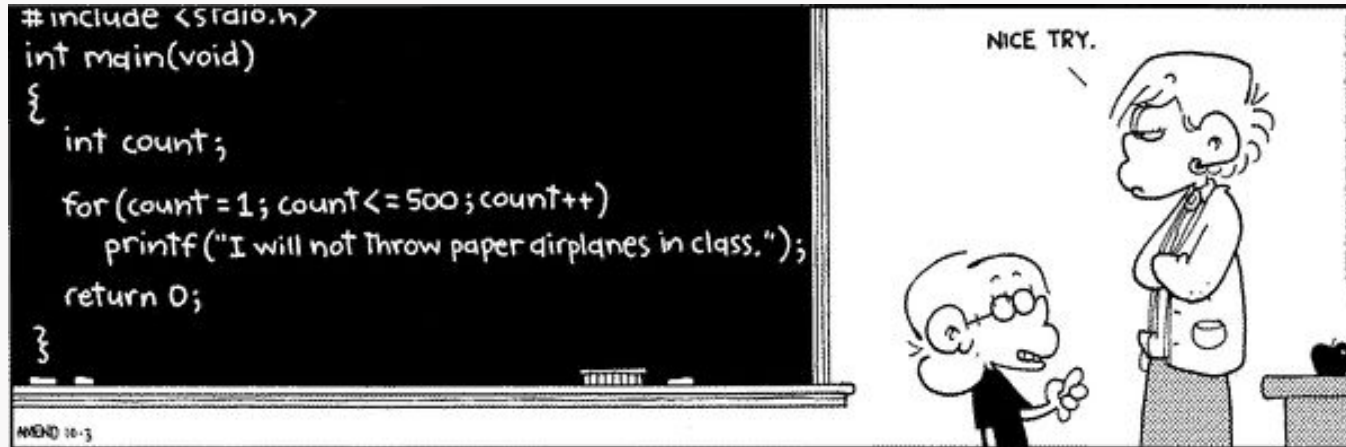


Objectives for this session

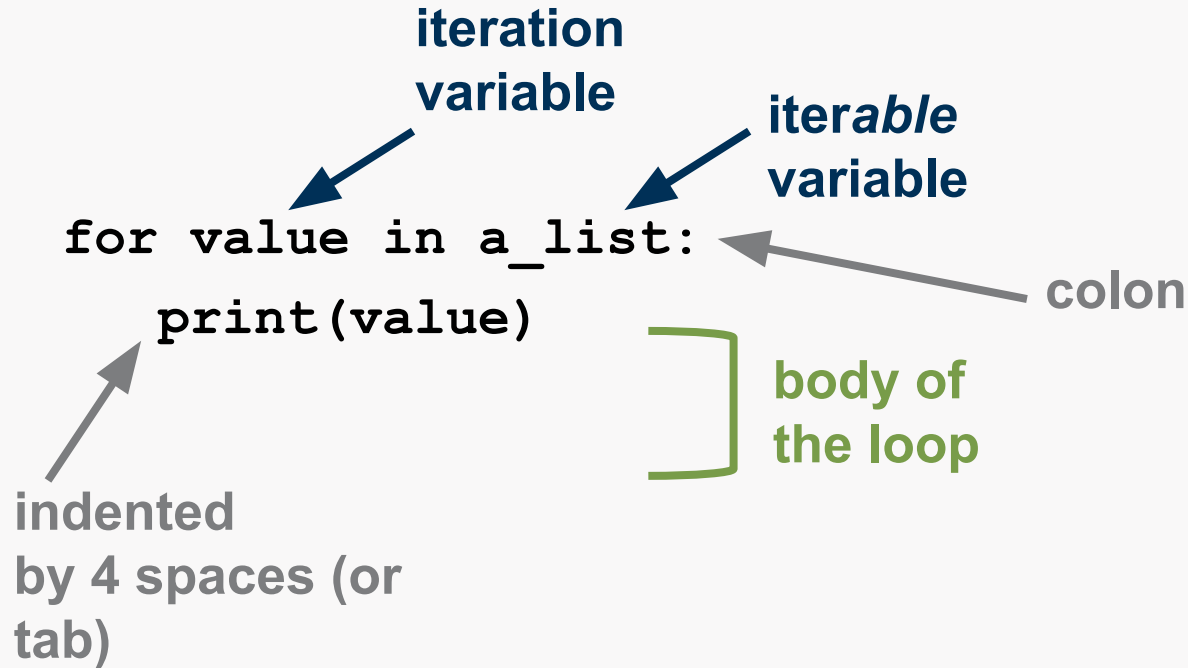
- Write a **for loop** to iterate over elements in an object
- Recognize **Booleans** & write conditional logic statements
- Test **conditional statements** in Python
- Recognize **function** syntax & write a simple function

A **loop** is a procedure to repeat a piece of code.
(another way of saying this is that it iterates through code)

- Loops enable you to re-run blocks of code as many times as you need.
- Python has two main ways to run loops: `for` & `while`



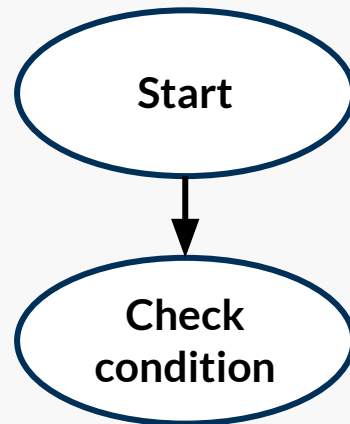
for loop syntax



A **for loop** is a procedure to repeat code for every element in a sequence.

for loop syntax

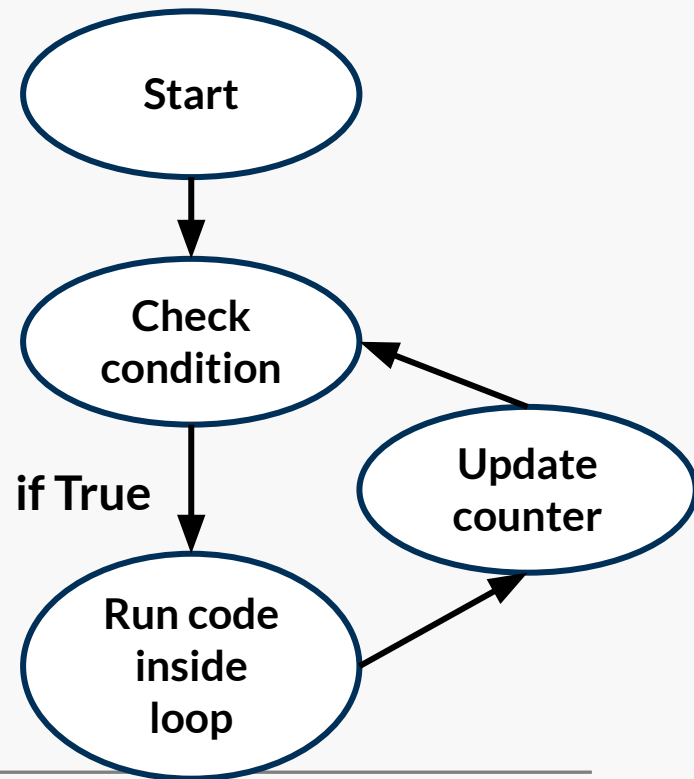
```
a_list = [1,2,3]  
for value in a_list:  
    print(value)
```



for loop syntax

```
a_list = [1,2,3]
for value in a_list:
    print(value)
```

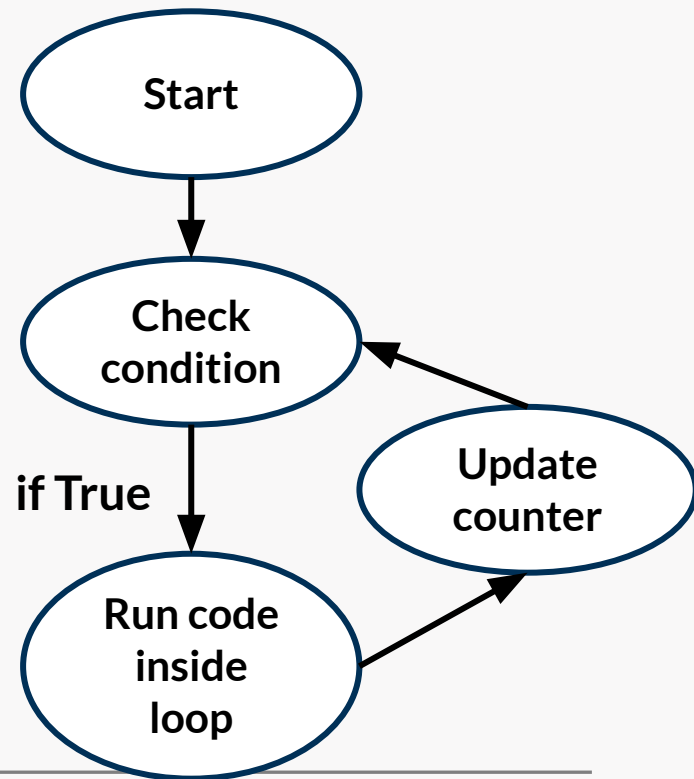
1 | output



for loop syntax

```
a_list = [1,2,3]
for value in a_list:
    print(value)
```

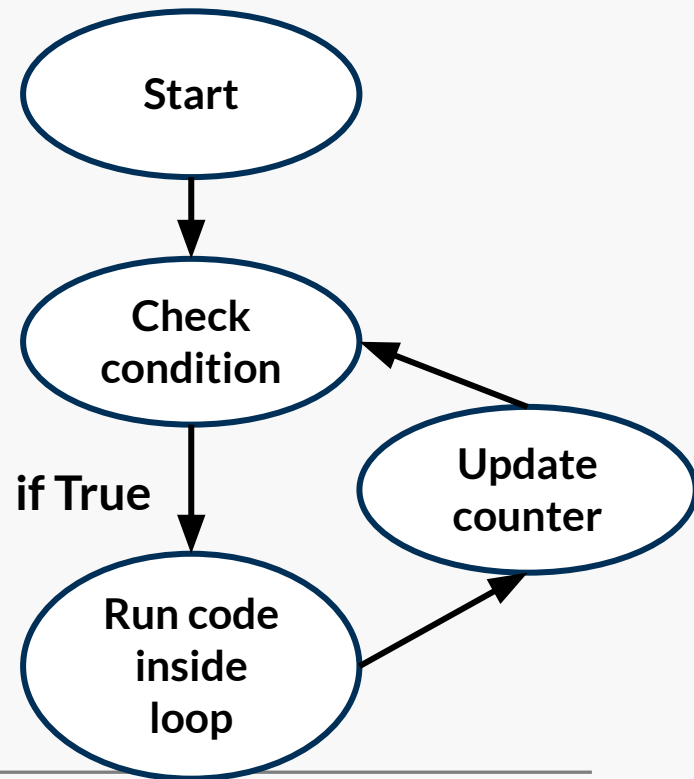
1 |
2 | output



for loop syntax

```
a_list = [1,2,3]
for value in a_list:
    print(value)
```

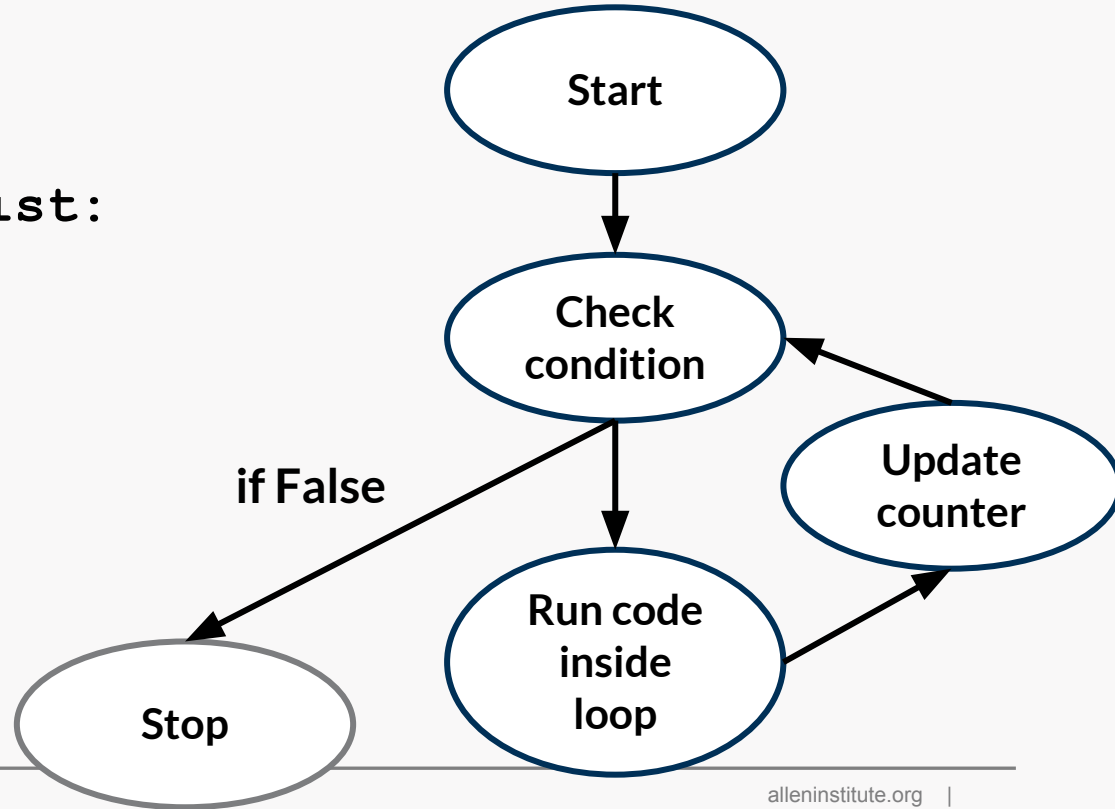
1 |
2 | output
3 |



for loop syntax

```
a_list = [1,2,3]
for value in a_list:
    print(value)
```

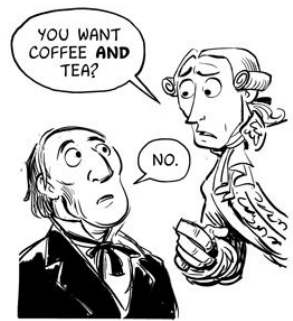
1 |
2 | output
3 |



Basic conditional operators in Python

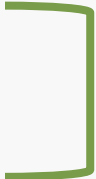
Symbol	Operation	Usage	Outcome
<code>==</code>	Is equal to	<code>10==5*2</code>	True
<code>!=</code>	Is not equal to	<code>10 != 5*2</code>	False
<code>></code>	Is greater than	<code>10 > 2</code>	True
<code><</code>	Is less than	<code>10 < 2</code>	False
<code>>=</code>	Greater than <i>or</i> equal to	<code>10 >= 10</code>	True
<code><=</code>	Less than <i>or</i> equal to	<code>10 <= 10</code>	True

Boolean variables
store **True** (1) or
False (0) and are the
basis of all computer
operations.



if statements syntax

`if condition:`  you need a colon here!

indented
by 4 spaces
(or tab)  `print('condition met')`
`print('nice work.')`  block

`print('not in the block')`

if/else statement syntax

```
if condition:
```

```
    print('condition met')
```

```
    print('nice work.')
```

```
else:
```

```
    print('condition not met')
```



**you need a
colon here!**

Functions

Functions are pieces of code that are designed to do one task

Functions take in inputs, process those inputs, and possibly return an output.

Python has built-in functions, but we can also write our own!

function syntax

function
name

def function(value) : ← colon

print(value)

function
body

indented
by 4 spaces
(or tab)

function syntax

input arguments (these can be variables or default arguments)

```
def function(b):
```

```
    a = b**2
```

```
    return a
```

return to retrieve a variable outside of a function (*what happens in the function stays in the function*)
ALSO ENDS THE FUNCTION!

call to function giving it the argument and saving the returned variable as a

```
a = function(6)
```


Resources

[Intro. to Comp. Sci. & OOP: Python · Cogniterra](#)

[Plotting and Programming in Python: For Loops](#)

[Plotting and Programming in Python: Conditionals](#)

[Whirlwind Tour of Python: Control Flow](#)

[Merely Useful Functions](#)

[Python Tutorial: Functions](#)